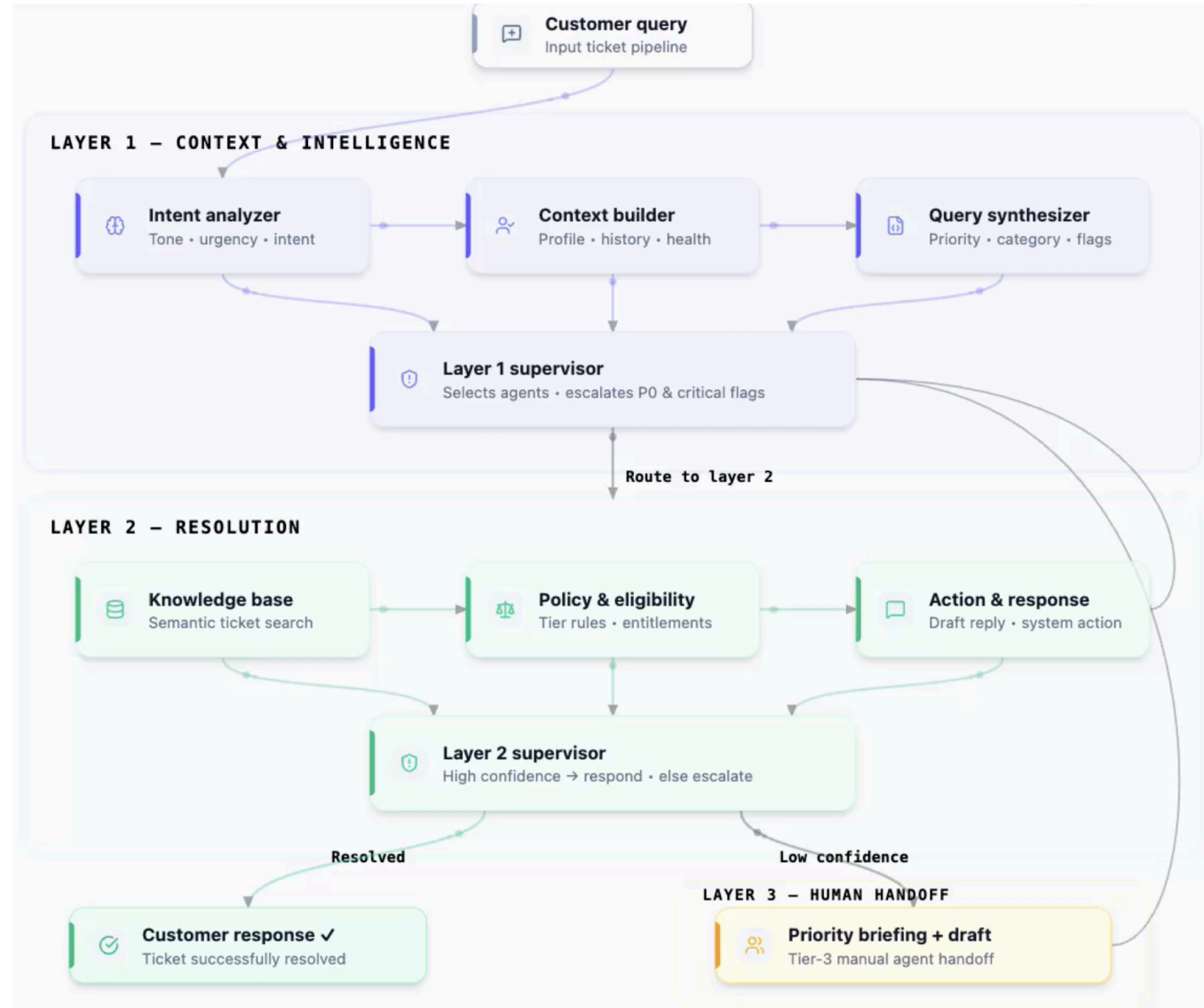


NexusDesk

Multi-Agent Customer Support System

LLM Backend Gemini 2.5 Flash	Architecture 3-Layer Hierarchical Agents
Vector Store ChromaDB + SentenceTransformers	Test Coverage Full Suite — All Agents Mocked

Shivansh · FlowDesk SaaS Platform · 2026



1. Project Overview

NexusDesk is a production-grade multi-agent AI system built for FlowDesk, a fictional project management SaaS. It receives a raw customer query, processes it through a hierarchical pipeline of specialized AI agents, and either resolves it autonomously or escalates it to a human agent with a fully pre-packaged briefing and a pre-written response draft.

Why This Project Was Built

The deliberate goal: to bridge the gap between LangChain and LangGraph by building a multi-agent system intentionally without LangGraph — and feeling exactly why an orchestration framework exists. NexusDesk has no cycles, no parallel fan-outs, and no dynamic agent spawning. It is a clean linear flow with two routing decisions. And yet it still demands significant orchestration code to handle correctly: routing logic, fallback paths, non-fatal error handling, state passing, and layered supervision decisions. That is the exact pain point LangGraph was built to solve.

Also to practice professionalism which reflects in the folder structure, code and error handling

System at a Glance

Property	Value
Domain	Customer Support Automation (FlowDesk SaaS)
Architecture	3-Layer Hierarchical Multi-Agent Pipeline
Total Agents	8 (3 in Layer 1, 3 in Layer 2, 2 Supervisors)
LLM	Gemini 2.5 Flash via langchain-google-genai
Orchestration	LangChain LCEL chains + structured output (no LangGraph)
Vector Store	ChromaDB with SentenceTransformers local embeddings
Data Validation	Pydantic v2 — strict models for every agent I/O
Test Suite	pytest + unittest.mock — all LLM calls fully patched
Logging	Python logging module — no bare print() statements

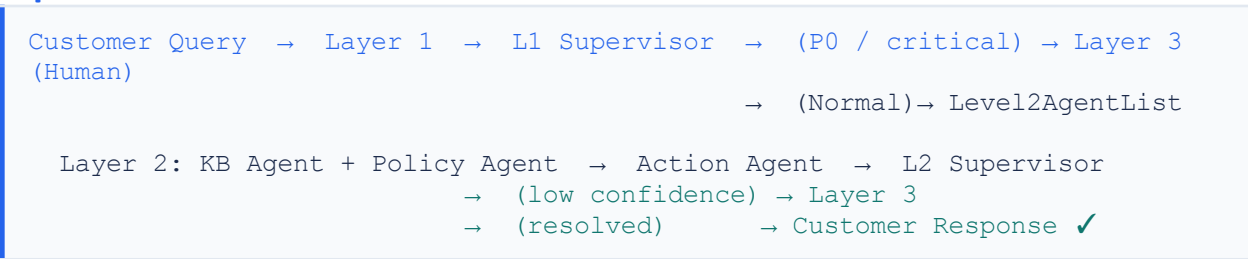
Core Problem Being Solved

- ◆ Customer queries arrive with varying intent, urgency, and emotional tone — hard to route without context
- ◆ Simple rule-based systems cannot distinguish P0 security incidents from standard billing questions
- ◆ Resolution requires cross-referencing knowledge base tickets AND company policy simultaneously
- ◆ Human agents need a structured briefing + pre-drafted response when auto-resolution is not possible
- ◆ System must remain functional even when individual agents fail — no single point of failure

2. System Architecture (starting with Layer 1)

The pipeline processes every incoming support query through two AI layers before optionally routing to a human. Each layer has a supervisor that makes routing decisions based on the outputs of its agents.

Pipeline Flow



Layer 1 — Context & Intelligence

Three agents run sequentially. Each agent passes its output to the next, building up a rich structured context object.

Agent 1 — Intent Analyzer — <i>Parses the raw customer message</i>	
Inputs	Raw customer query string
Outputs	Intent model: emotional_tone, urgency, literal_query, actual_intent, already_tried
Model	Gemini 2.5 Flash → structured output via LangChain LCEL

Agent 2 — Context Builder — <i>Retrieves and synthesizes customer history</i>	
Inputs	customer_id → ChromaDB lookup (customers + tickets collections)
Outputs	CustomerContext: subscription_tier, account_health, past_tickets, context_summary
Model	Gemini 2.5 Flash + ChromaDB metadata query

Agent 3 — Query Synthesizer — <i>Merges intent + context into a unified enriched package</i>	
Inputs	Intent model + CustomerContext model
Outputs	EnrichedQueryPackage: category, priority_level, escalation_flags, suggested_layer2_agents, enriched_brief
Model	Gemini 2.5 Flash → structured EnrichedQueryPackage output

L1 Supervisor Logic: Pure Python — no LLM call. Routes to Layer 3 if priority_level == "P0" OR escalation_flags contains a critical flag (e.g. "security_incident"). Otherwise routes to Layer 2 with the suggested agent list.

3. Layer 2 — Resolution Pipeline

Layer 2 runs three agents using the EnrichedQueryPackage from Layer 1. KB and Policy agents run based on the suggested list; the Action Agent always runs on their combined outputs.

Knowledge Base Agent — Semantic search over resolved support tickets	
Inputs	actual_intent, query_category, subscription_tier
Outputs	KBResult: found, confidence, matched_ticket_ids, resolution_steps, resolution_summary, recommended_action, caveat
Model	ChromaDB metadata-filtered semantic query → Gemini 2.5 Flash synthesis

Policy & Eligibility Agent — Checks company policy for customer eligibility	
Inputs	actual_intent, query_category, subscription_tier
Outputs	PolicyResult: found, confidence, matched_policy_ids, is_eligible, policy_summary, recommended_action, caveat
Model	ChromaDB policy collection query (JSON-encoded tier lists) → Gemini 2.5 Flash synthesis

Action & Response Agent — Generates the resolution and customer-facing message	
Inputs	EnrichedQueryPackage + KBResult + PolicyResult (either or both)
Outputs	ActionResult: resolution_confidence, requires_human, system_action, response_draft, reasoning
Model	Gemini 2.5 Flash with full context injection

L2 Supervisor Logic: Pure Python — no LLM call. Routes to Layer 3 if resolution_confidence == "low" OR requires_human == True. Otherwise passes resolved response + system_action to caller.

4. Layer 3 — Human Handoff

Layer 3 is not an AI agent — it is a packaging function that prepares a structured briefing for a human support agent. In production, this would push to a CRM, ticket queue, or Slack workflow.

Output Field	Description
status	"escalated_to_human"

Output Field	Description
customer_id	Customer identifier
customer_name	Full name for personalisation
priority	P0 / P1 / P2 for queue routing
reason	Why auto-resolution failed or was skipped
enriched_brief	Full synthesized brief from Layer 1
draft_for_agent	Pre-written response the human can use or discard

5. Data Models (Pydantic v2)

Every agent I/O is strictly typed using Pydantic v2 models. There are no raw dicts passed between agents — structured output is enforced via LangChain's `.with_structured_output()` binding.

Layer 1 Models

Model	Key Fields	Purpose
Intent	emotional_tone, urgency, literal_query, actual_intent, already_tried	Output of Agent 1
CustomerContext	customer_id, customer_name, company, subscription_tier, team_size, account_health, past_tickets, context_summary	Output of Agent 2
EnrichedQueryPackage	All Intent + Context fields + query_category, priority_level, escalation_flags, suggested_layer2_agents, enriched_brief	Output of Agent 3 — primary data carrier across all layers

Layer 2 Models

Model	Key Fields	Purpose
KBResult	found, confidence, matched_ticket_ids, resolution_steps, resolution_summary, recommended_action, caveat	Output of KB Agent

Model	Key Fields	Purpose
PolicyResult	found, confidence, matched_policy_ids, is_eligible, policy_summary, recommended_action, caveat	Output of Policy Agent
ActionResult	resolution_confidence, requires_human, system_action, response_draft, reasoning	Output of Action Agent — final resolution payload

ChromaDB Collections

Three separate ChromaDB collections are used, each queried with metadata filtering before semantic ranking:

Collection	Contents	Query Strategy
customers	Customer profiles with account status, subscription tier, team size, account health	Exact-match lookup by customer_id via .get()
tickets	Resolved support tickets with resolution steps	Semantic query filtered by category + subscription_tier
policies	Company policy documents with tier applicability	Semantic query; tier lists stored as JSON strings (ChromaDB limitation — no native list metadata)

6. Error Handling Strategy

NexusDesk implements a layered, non-crashing fault tolerance strategy. The goal is graceful degradation at every stage rather than a single unhandled exception that breaks the entire pipeline.

Failure Mode	Trigger	System Response
Layer 1 — Domain Error	ValueError from context builder (inactive account, customer not found, empty query)	Returns {"status": "error"} — does not escalate or crash
Layer 1 — Unexpected	Any unhandled exception in the L1 pipeline	Returns {"status": "error"} — does not escalate
Layer 2 — KB Agent crash	RuntimeError (e.g. ChromaDB unavailable)	Non-fatal — Action Agent runs on Policy result alone
Layer 2 — Policy Agent crash	RuntimeError	Non-fatal — Action Agent runs on KB result alone

Failure Mode	Trigger	System Response
Layer 2 — Both agents crash	Both KB and Policy agents fail	Immediate human handoff — cannot generate resolution without any upstream data
Layer 2 — Action Agent crash	Unhandled exception in Action Agent	Human handoff with empty response draft
L2 Supervisor — Low confidence	resolution_confidence == "low"	Human handoff with the AI-drafted response passed to human
L2 Supervisor — Requires human	requires_human == True	Human handoff with the AI-drafted response passed to human
L1 Supervisor — P0 priority	priority_level == "P0"	Immediate Layer 3 handoff — Layer 2 never runs
L1 Supervisor — Critical flag	escalation_flags contains critical flag	Immediate Layer 3 handoff — Layer 2 never runs

7. Test Suite

All 8 agents and the full orchestration pipeline are covered by unit tests using Python's unittest.mock. LLM API calls and ChromaDB queries are fully patched — tests run instantly with no API key and zero cost.

Test File	What It Covers
test_intent_analyzer.py	Verifies LangChain chain is invoked once; checks emotional_tone and urgency fields
test_context_builder.py	Mocks ChromaDB collections + chain; tests inactive account ValueError path
test_query_synthesizer.py	Mocks chain; verifies query_category and priority_level in output
test_layer1_supervisor.py	Pure logic — no mock needed; tests P2→layer2, P0→layer3, critical flag→layer3
test_kb_agent.py	Mocks collection.query() + chain; tests happy path and empty-intent ValueError
test_policy_agent.py	Mocks collection.query() with JSON-encoded tier lists; tests eligibility and ValueError
test_action_agent.py	Mocks chain; tests happy path and ValueError when both upstream inputs are None

Test File	What It Covers
test_layer2_supervisor.py	Pure logic — no mock; tests resolve, low-confidence escalation, requires_human escalation
test_main.py	End-to-end orchestration tests; patches all 6 agents; covers 7 scenarios including non-fatal failures

8. Tech Stack & Key Engineering Decisions

Technology Stack

Layer	Technology	Library / Notes
LLM	Gemini 2.5 Flash	langchain-google-genai
Orchestration	LangChain LCEL chains	langchain-core
Vector Store	ChromaDB	chromadb (direct library, not LangChain wrapper)
Embeddings	SentenceTransformers	sentence-transformers (local, no API key needed)
Validation	Pydantic v2	pydantic
Testing	pytest + unittest.mock	pytest
Logging	Python standard logging module	Built-in — no bare print() statements

Key Engineering Decisions

ChromaDB over FAISS

- ◆ ChromaDB supports metadata filtering on queries (WHERE clause conditions), which was essential for filtering tickets by category and subscription_tier before semantic ranking.
- ◆ FAISS is a flat vector index — it does not support metadata-conditional retrieval natively.
- ◆ ChromaDB does not support list-type metadata natively, so applicable_tiers is stored as a JSON-encoded string and filtered using the \$contains operator.

Direct chromadb Library over LangChain's Chroma Wrapper

- ◆ LangChain's Chroma abstraction hides fine-grained control over query parameters, metadata schema, and batch operations.
- ◆ Using the chromadb library directly allows explicit batch upserts (optimized by the library) and precise where-clause construction.

Module-Level LLM Client Instantiation

- ◆ LLM clients and chains are instantiated once at module level (prefixed with `_` to signal internal use), not inside the function body.
- ◆ Instantiating inside a function means re-creating the LLM client on every single call — unnecessary and costly in production.

Non-Fatal Agent Failures in Layer 2

- ◆ KB Agent and Policy Agent failures are caught individually and are non-fatal by design.
- ◆ The Action Agent can synthesize a resolution from either result alone — requiring both to succeed would be an unnecessary hard dependency.
- ◆ Only if both fail does the system fall back to human handoff, and the guard check is explicit in the orchestration code.

Structured Output Enforcement

- ◆ Every agent uses LangChain's `.with_structured_output(PydanticModel)` binding — the LLM is constrained to return valid structured JSON matching the Pydantic schema.
- ◆ This eliminates the need for manual JSON parsing, validation, or error correction of LLM outputs.

Separation of Layer 1 and Layer 2

- ◆ Layer 1 is entirely about understanding and enriching context — it never touches the knowledge base or policy store.
- ◆ Layer 2 is entirely about resolution using the enriched package — it never re-analyzes intent or rebuilds context.
- ◆ This clean separation means each layer can be independently replaced, tested, or scaled.

9. Project Structure

The project follows a production-grade folder structure adapted from industry standards — each concern is isolated into its own module.

```
NexusDeskBackend/  
├─ main.py                # Full orchestration pipeline (entry  
point)  
├─ requirements.txt        # 6 dependencies  
├─ pytest.ini              # Test config (suppress deprecation  
warnings)  
├─ src/  
│   └─ agents/  
│       └─ level_1/  
│           └─ intent_analyzer.py    # Agent 1
```

```

├── context_builder.py      # Agent 2
├── query_synthesizer.py    # Agent 3
├── level1_supervisor.py    # L1 routing supervisor
├── level_2/
│   ├── kb_agent.py        # Knowledge Base Agent
│   ├── policy_agent.py    # Policy & Eligibility Agent
│   ├── action_agent.py    # Action & Response Agent
│   └── level2_supervisor.py # L2 routing supervisor
├── models/                # Pydantic v2 models (one per agent)
├── I/O)
│   ├── prompts/          # LangChain prompt templates (one per
│   │   └── agent)
│   └── utils/            # Logger, data preprocessing,
│       ├── embedding pipeline
│       ├── data/
│       │   ├── knowledge_base/
│       │   │   ├── profiles, tickets)
│       │   │   └── processed/
│       │       └── ChromaDB upsert
│       ├── chromadb/      # Persistent ChromaDB vector store
│       ├── tests/        # Full unit test suite (9 files)
│       └── logs/         # Runtime logs (.gitkeep for empty
└── dir)

```

Running the Project

Install Dependencies

```

pip install langchain-google-genai langchain-core chromadb
sentence-transformers pydantic pytest

```

Embed Data into ChromaDB

```

python src/utils/data_preprocessing.py
python src/utils/embedding_function.py

```

Run the Pipeline

```

python main.py CUST_001 "I was charged twice and I have a demo tomorrow"

```

Run Tests

```

python -m pytest tests/ -v -s

```

10. What This Project Demonstrates

Multi-Agent System Design

- ◆ Designed and implemented a 3-layer hierarchical agent pipeline with 8 specialized agents

- ◆ Each agent has a single, well-defined responsibility — strict separation of concerns across layers
- ◆ Supervisor pattern: two pure-logic routing supervisors make escalation decisions without LLM calls
- ◆ Agent I/O contracts enforced via Pydantic v2 — no raw dict passing between agents

LangChain & LLM Engineering

- ◆ LangChain LCEL chains with structured output binding for every LLM-powered agent
- ◆ Gemini 2.5 Flash used via langchain-google-genai with structured Pydantic output enforcement
- ◆ Module-level LLM instantiation (not per-call) — production-aware performance pattern
- ◆ Prompt templates isolated per agent in a dedicated prompts/ module

Vector Store & RAG

- ◆ ChromaDB chosen over FAISS for its metadata filtering capability — filtered RAG, not naive semantic search
- ◆ Direct chromadb library usage for fine-grained query control and batch upsert operations
- ◆ SentenceTransformers for local embeddings — no external embedding API required
- ◆ JSON-encoded list metadata workaround for ChromaDB limitation on list-type fields

Production-Grade Engineering

- ◆ Layered fault tolerance: non-fatal agent failures, graceful degradation, no unhandled crashes
- ◆ Python logging module throughout — no bare print() statements
- ◆ _ prefix convention for internal module-level variables following Python standards
- ◆ Domain errors (ValueError) separated from unexpected errors (Exception) with distinct handling

Testing Discipline

- ◆ Full unit test suite using pytest + unittest.mock — all LLM API calls fully patched
- ◆ 9 test files covering all 8 agents individually and the full end-to-end pipeline
- ◆ Tests cover happy paths, escalation paths, error paths, and non-fatal failure paths
- ◆ Tests run instantly with no API key and zero cost — infrastructure-independent

The Core Takeaway

NexusDesk is not a complex graph — it has no cycles, no parallel fan-outs, no dynamic agent spawning. It is a clean linear pipeline with two routing decisions. And it still required significant orchestration code to handle correctly. That is precisely the argument for frameworks like LangGraph. NexusDesk was built to make that argument tangible — not just understood, but felt.